

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - Database Design and Connectivity

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Course Code:	DSA0501	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	AT1 – Database Design and Connectivity
Weightage:	Total Marks: 20 Marks	CO2 Statement:	Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)
Student Name:	SATHISH S	Reg. No.:	192324318L
Section / Batch:		Date of Submission:	
Faculty In-charge:	K Veena devi	Project Mode:	Individual Submission

Code of Conduct

I AM Sathish s / 192324318L certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature; Sathish s

CO2 – AT1 – Database Design and Connectivity: Solutions

Question 1: Student Database Design and Connectivity (10 Marks)

Problem Statement: A college wants to automate the management of student records. Design an SQLite database containing a Student table with appropriate fields and a primary key. Write a Python program to connect to the SQLite database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

1. Requirement Analysis (4 Marks)

- Entities identified: Student
- Attributes: student_id (INTEGER, PK, AUTOINCREMENT), name (TEXT, NOT NULL), age (INTEGER), department (TEXT), gpa (REAL).
- Functional Goal: Automate student record maintenance, ensure data integrity, and allow automated listing of student data.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: Entity [Student] with key attribute (student_id#), composite/atomic attributes (name, age, department, gpa).
- Relational Schema: Student(student_id [PK], name, age, department, gpa)

3. Table Design & Normalization (4 Marks)

- 1NF: All attributes contain atomic values (no multivalued attributes).
- 2NF: Table is in 1NF and all non-key attributes fully functionally depend on the primary key (student_id).
- 3NF: No transitive dependencies exist.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def manage_student_db():
    conn = sqlite3.connect("student_management.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Student (
            student_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            age INTEGER,
            department TEXT,
            gpa REAL
        )
    """)

    students = [
        ('Alice Smith', 20, 'Computer Science', 3.85),
        ('Bob Jones', 21, 'Electrical', 3.60),
        ('Charlie Brown', 19, 'Mechanical', 3.40),
        ('Diana Prince', 22, 'Civil', 3.90),
        ('Ethan Hunt', 20, 'Data Science', 3.75)
    ]
```

```

cursor.executemany("""
    INSERT INTO Student (name, age, department, gpa)
    VALUES (?, ?, ?, ?)
""", students)
conn.commit()

print("--- Stored Student Records ---")
cursor.execute("SELECT * FROM Student")
records = cursor.fetchall()
for row in records:
    print(f"ID: {row[0]} | Name: {row[1]} | Age: {row[2]} | Dept: {row[3]} | GPA: {row[4]}")

conn.close()

if __name__ == "__main__":
    manage_student_db()

```

Sample Execution Output:

```

--- Stored Student Records ---
ID: 1 | Name: Alice Smith | Age: 20 | Dept: Computer Science | GPA: 3.85
ID: 2 | Name: Bob Jones | Age: 21 | Dept: Electrical | GPA: 3.6
ID: 3 | Name: Charlie Brown | Age: 19 | Dept: Mechanical | GPA: 3.4
ID: 4 | Name: Diana Prince | Age: 22 | Dept: Civil | GPA: 3.9
ID: 5 | Name: Ethan Hunt | Age: 20 | Dept: Data Science | GPA: 3.75

```

Question 2: Library Management Database (10 Marks)

Problem Statement: A library plans to maintain information about its books using an SQLite database. Design a Book table containing suitable attributes such as Book ID, Title, Author, Publisher, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

1. Requirement Analysis (4 Marks)

- Entities: Book
- Attributes: book_id (INTEGER, PK), title (TEXT), author (TEXT), publisher (TEXT), price (REAL).
- Functional Goal: Track library books, execute price updates, and fetch updated book details.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: Entity [Book] with primary key book_id.
- Relational Schema: Book(book_id [PK], title, author, publisher, price)

3. Table Design & Normalization (4 Marks)

- 1NF, 2NF, 3NF satisfied as attributes are non-divisible, depend entirely on book_id, and have zero non-key transitive dependencies.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```

import sqlite3

def library_db_operations():
    conn = sqlite3.connect("library.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Book (
            book_id INTEGER PRIMARY KEY,
            title TEXT NOT NULL,
            author TEXT NOT NULL,
            publisher TEXT,
            price REAL
        )
    """)

    books = [
        (101, 'Python Programming', 'John Zelle', 'Franklin', 45.00),
        (102, 'Data Science Essentials', 'Joel Grus', 'O Reilly', 55.00),
        (103, 'Database Systems', 'Ramez Elmasri', 'Pearson', 65.00)
    ]
    cursor.executemany("INSERT OR REPLACE INTO Book VALUES (?, ?, ?, ?, ?)", books)
    conn.commit()

    print("Before Update:")
    cursor.execute("SELECT * FROM Book WHERE book_id = 101")
    print(cursor.fetchone())

    # Update Price
    new_price = 49.99
    cursor.execute("UPDATE Book SET price = ? WHERE book_id = ?", (new_price, 101))
    conn.commit()

    print("\nAfter Price Update (Book ID 101):")
    cursor.execute("SELECT * FROM Book WHERE book_id = 101")
    print(cursor.fetchone())

    conn.close()

if __name__ == "__main__":
    library_db_operations()

```

Sample Execution Output:

```

Before Update:
(101, 'Python Programming', 'John Zelle', 'Franklin', 45.0)

After Price Update (Book ID 101):
(101, 'Python Programming', 'John Zelle', 'Franklin', 49.99)

```

Question 3: Employee and Department Database Design (10 Marks)

Problem Statement: A company wants to store employee information and department details in an SQLite database. Design two related tables, Department and Employee, by identifying appropriate primary and foreign

keys. Write a Python program to connect to the database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using a SQL JOIN query.

1. Requirement Analysis (4 Marks)

- Entities: Department, Employee
- Attributes:
 - Department: dept_id (PK), dept_name, location
 - Employee: emp_id (PK), emp_name, salary, dept_id (FK -> Department.dept_id)
- Relationship: One Department has Many Employees (1:N).

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: [Department] (1) ----- <Employs> ----- (N) [Employee]
- Relational Schema:
 - Department(dept_id [PK], dept_name, location)
 - Employee(emp_id [PK], emp_name, salary, dept_id [FK])

3. Table Design & Normalization (4 Marks)

- Split into two relations to eliminate redundant department names and achieve 3NF compliance.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def employee_department_system():
    conn = sqlite3.connect("company.db")
    cursor = conn.cursor()

    cursor.execute("PRAGMA foreign_keys = ON;")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Department (
            dept_id INTEGER PRIMARY KEY,
            dept_name TEXT NOT NULL,
            location TEXT
        )
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Employee (
            emp_id INTEGER PRIMARY KEY,
            emp_name TEXT NOT NULL,
            salary REAL,
            dept_id INTEGER,
            FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
        )
    """)

    departments = [
        (10, 'Human Resources', 'New York'),
        (20, 'Engineering', 'San Francisco'),
        (30, 'Marketing', 'Chicago')
    ]
    cursor.executemany("INSERT OR REPLACE INTO Department VALUES (?, ?, ?)", departments)
```

```

employees = [
    (1001, 'Alice', 75000, 20),
    (1002, 'Bob', 50000, 10),
    (1003, 'Charlie', 85000, 20),
    (1004, 'David', 60000, 30)
]
cursor.executemany("INSERT OR REPLACE INTO Employee VALUES (?, ?, ?, ?)", employees)
conn.commit()

print("--- Employee Details with Department (SQL JOIN) ---")
query = """
    SELECT e.emp_name, d.dept_name, d.location
    FROM Employee e
    JOIN Department d ON e.dept_id = d.dept_id
"""
cursor.execute(query)
for emp_name, dept_name, loc in cursor.fetchall():
    print(f"Employee: {emp_name:<10} | Department: {dept_name:<20} | Location: {loc}")

conn.close()

if __name__ == "__main__":
    employee_department_system()

```

Sample Execution Output:

```

--- Employee Details with Department (SQL JOIN) ---
Employee: Alice      | Department: Engineering      | Location: San Francisco
Employee: Bob        | Department: Human Resources   | Location: New York
Employee: Charlie    | Department: Engineering      | Location: San Francisco
Employee: David       | Department: Marketing         | Location: Chicago

```

Question 4: Hospital Patient Database (10 Marks)

Problem Statement: A hospital requires a database to manage patient information. Design an SQLite database with a Patient table containing suitable fields and constraints. Write a Python program to connect to the database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

1. Requirement Analysis (4 Marks)

- Entities: Patient
- Attributes: patient_id (PK), name, age, contact, status.
- Operations: Insert, Update contact, Delete discharged patient, Retrieve active records.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: Entity [Patient] with constraints (patient_id PK, contact NOT NULL).
- Relational Schema: Patient(patient_id [PK], name, age, contact, status)

3. Table Design & Normalization (4 Marks)

- Full compliance with 1NF, 2NF, and 3NF.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def hospital_patient_system():
    conn = sqlite3.connect("hospital.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Patient (
            patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            age INTEGER,
            contact TEXT NOT NULL,
            status TEXT
        )
    """)

    patients = [
        ('John Doe', 45, '9876543210', 'Admitted'),
        ('Jane Smith', 32, '8765432109', 'Discharged'),
        ('Mark Ruffalo', 50, '7654321098', 'Admitted')
    ]
    cursor.executemany("INSERT INTO Patient (name, age, contact, status) VALUES (?, ?, ?, ?)",
patients)
    conn.commit()

    # Update Contact Number for John Doe (ID: 1)
    cursor.execute("UPDATE Patient SET contact = ? WHERE patient_id = ?", ('9998887770', 1))

    # Delete Discharged Patient
    cursor.execute("DELETE FROM Patient WHERE status = ?", ('Discharged',))
    conn.commit()

    print("--- Current Active Hospital Patients ---")
    cursor.execute("SELECT * FROM Patient")
    for row in cursor.fetchall():
        print(f"ID: {row[0]} | Name: {row[1]} | Age: {row[2]} | Contact: {row[3]} | Status: {row[4]}")

    conn.close()

if __name__ == "__main__":
    hospital_patient_system()
```

Sample Execution Output:

```
--- Current Active Hospital Patients ---
ID: 1 | Name: John Doe | Age: 45 | Contact: 9998887770 | Status: Admitted
ID: 3 | Name: Mark Ruffalo | Age: 50 | Contact: 7654321098 | Status: Admitted
```

Question 5: Online Course Registration System (10 Marks)

Problem Statement: An online learning platform needs to maintain information about students and the courses they have registered for. Design two related SQLite tables, Student and Course_Registration, using appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample records, and retrieve the names of students along with the courses they have registered for using an SQL JOIN operation.

1. Requirement Analysis (4 Marks)

- Entities: Student, Course_Registration
- Attributes:
 - Student: student_id (PK), student_name, email
 - Course_Registration: reg_id (PK), student_id (FK), course_name, reg_date
- Relationship: 1 Student to Many Course Registrations.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: [Student] (1) ----- <Registers> ----- (N) [Course_Registration]
- Schema:
 - Student(student_id [PK], student_name, email)
 - Course_Registration(reg_id [PK], student_id [FK], course_name, reg_date)

3. Table Design & Normalization (4 Marks)

- Tables separated to prevent multi-valued course entries per student (3NF achieved).

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def online_course_system():
    conn = sqlite3.connect("course_reg.db")
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Student (
            student_id INTEGER PRIMARY KEY,
            student_name TEXT NOT NULL,
            email TEXT UNIQUE
        )
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Course_Registration (
            reg_id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id INTEGER,
            course_name TEXT NOT NULL,
            reg_date TEXT,
            FOREIGN KEY (student_id) REFERENCES Student(student_id)
        )
    """)

    students = [
        (1, 'Emma Watson', 'emma@example.com'),
        (2, 'Liam Neeson', 'liam@example.com')
    ]
    cursor.executemany("INSERT OR REPLACE INTO Student VALUES (?, ?, ?)", students)
```



```

regs = [
    (1, 'Data Science 101', '2026-03-01'),
    (1, 'SQL & Databases', '2026-03-02'),
    (2, 'Machine Learning', '2026-03-05')
]
cursor.executemany("INSERT INTO Course_Registration (student_id, course_name, reg_date)
VALUES (?, ?, ?)", regs)
conn.commit()

print("--- Student Course Registrations ---")
query = """
    SELECT s.student_name, r.course_name, r.reg_date
    FROM Student s
    JOIN Course_Registration r ON s.student_id = r.student_id
"""
cursor.execute(query)
for name, course, date in cursor.fetchall():
    print(f"Student: {name:<15} | Course Registered: {course:<20} | Date: {date}")

conn.close()

if __name__ == "__main__":
    online_course_system()

```

Sample Execution Output:

```

--- Student Course Registrations ---
Student: Emma Watson      | Course Registered: Data Science 101      | Date: 2026-03-01
Student: Emma Watson      | Course Registered: SQL & Databases        | Date: 2026-03-02
Student: Liam Neeson      | Course Registered: Machine Learning      | Date: 2026-03-05

```

Question 6: Banking Account Management System (10 Marks)

Problem Statement: A bank wants to maintain customer account information using an SQLite database. Design an appropriate database containing an Account table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert customer account details, update the account balance of a specified customer, and display all customer records.

1. Requirement Analysis (4 Marks)

- Entities: Account
- Attributes: acc_no (PK), cust_name, acc_type, balance (CHECK >= 0).
- Operations: Insert accounts, Update balance, Retrieve all details.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: Entity [Account] with attribute constraints (balance CHECK >= 0).
- Relational Schema: Account(acc_no [PK], cust_name, acc_type, balance)

3. Table Design & Normalization (4 Marks)

- Relational schema is in 3NF as all non-key fields depend directly and solely on acc_no.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def banking_system():
    conn = sqlite3.connect("banking.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Account (
            acc_no INTEGER PRIMARY KEY,
            cust_name TEXT NOT NULL,
            acc_type TEXT CHECK(acc_type IN ('Savings', 'Current')),
            balance REAL CHECK(balance >= 0)
        )
    """)

    accounts = [
        (5001, 'Alex Green', 'Savings', 15000.50),
        (5002, 'Sophia White', 'Current', 45000.00)
    ]
    cursor.executemany("INSERT OR REPLACE INTO Account VALUES (?, ?, ?, ?)", accounts)
    conn.commit()

    # Update Balance
    cursor.execute("UPDATE Account SET balance = balance + 5000.00 WHERE acc_no = ?", (5001,))
    conn.commit()

    print("--- Updated Bank Account Records ---")
    cursor.execute("SELECT * FROM Account")
    for row in cursor.fetchall():
        print(f"Acc No: {row[0]} | Name: {row[1]} | Type: {row[2]} | Balance: ${row[3]:,.2f}")

    conn.close()

if __name__ == "__main__":
    banking_system()
```

Sample Execution Output:

```
--- Updated Bank Account Records ---
Acc No: 5001 | Name: Alex Green | Type: Savings | Balance: $20,000.50
Acc No: 5002 | Name: Sophia White | Type: Current | Balance: $45,000.00
```

Question 7: Inventory Management System (10 Marks)

Problem Statement: A retail store needs to maintain product inventory using SQLite. Design a Product table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to create the database, insert sample products, identify products with quantity less than 10, update their stock quantity, and display the updated inventory.

1. Requirement Analysis (4 Marks)

- Entities: Product
- Attributes: product_id (PK), product_name, category, quantity, unit_price.
- Functional Goal: Auto-identify low-stock items (< 10) and update inventory.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: Entity [Product] with primary key product_id.
- Relational Schema: Product(product_id [PK], product_name, category, quantity, unit_price)

3. Table Design & Normalization (4 Marks)

- Standard 3NF design with single-valued primary key.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def inventory_system():
    conn = sqlite3.connect("inventory.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Product (
            product_id INTEGER PRIMARY KEY,
            product_name TEXT NOT NULL,
            category TEXT,
            quantity INTEGER,
            unit_price REAL
        )
    """)

    products = [
        (201, 'Wireless Mouse', 'Electronics', 5, 25.00),
        (202, 'Mechanical Keyboard', 'Electronics', 15, 75.00),
        (203, 'USB Cable', 'Accessories', 8, 10.00)
    ]
    cursor.executemany("INSERT OR REPLACE INTO Product VALUES (?, ?, ?, ?, ?)", products)
    conn.commit()

    print("Low Stock Products (Qty < 10) before restock:")
    cursor.execute("SELECT product_name, quantity FROM Product WHERE quantity < 10")
    print(cursor.fetchall())

    # Restock low quantity items (+20)
    cursor.execute("UPDATE Product SET quantity = quantity + 20 WHERE quantity < 10")
    conn.commit()

    print("\n--- Updated Inventory List ---")
    cursor.execute("SELECT * FROM Product")
    for row in cursor.fetchall():
        print(f"ID: {row[0]} | Product: {row[1]:<20} | Cat: {row[2]:<12} | Qty: {row[3]} | Price: ${row[4]}")

    conn.close()
```

```
if __name__ == "__main__":  
    inventory_system()
```

Sample Execution Output:

```
Low Stock Products (Qty < 10) before restock:  
[('Wireless Mouse', 5), ('USB Cable', 8)]
```

```
--- Updated Inventory List ---
```

ID: 201	Product: Wireless Mouse	Cat: Electronics	Qty: 25	Price: \$25.0
ID: 202	Product: Mechanical Keyboard	Cat: Electronics	Qty: 15	Price: \$75.0
ID: 203	Product: USB Cable	Cat: Accessories	Qty: 28	Price: \$10.0

Question 8: Course and Faculty Management System (10 Marks)

Problem Statement: A university wants to maintain details of faculty members and the courses they teach. Design two related tables, Faculty and Course, using appropriate primary and foreign keys. Write a Python program to create the database, insert sample records, and display each faculty member along with the courses assigned using an SQL JOIN query.

1. Requirement Analysis (4 Marks)

- Entities: Faculty, Course
- Attributes:
 - Faculty: faculty_id (PK), faculty_name, designation
 - Course: course_id (PK), course_name, credits, faculty_id (FK)
- Relationship: 1 Faculty teaches 1 or more Courses.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: [Faculty] (1) ----- <Teaches> ----- (N) [Course]
- Schema:
 - Faculty(faculty_id [PK], faculty_name, designation)
 - Course(course_id [PK], course_name, credits, faculty_id [FK])

3. Table Design & Normalization (4 Marks)

- Relational normal forms up to 3NF are satisfied.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3  
  
def faculty_course_system():  
    conn = sqlite3.connect("university.db")  
    cursor = conn.cursor()  
    cursor.execute("PRAGMA foreign_keys = ON;")  
  
    cursor.execute("""  
        CREATE TABLE IF NOT EXISTS Faculty (  
            faculty_id INTEGER PRIMARY KEY,
```

```

        faculty_name TEXT NOT NULL,
        designation TEXT
    )
"""
)

cursor.execute("""
    CREATE TABLE IF NOT EXISTS Course (
        course_id TEXT PRIMARY KEY,
        course_name TEXT NOT NULL,
        credits INTEGER,
        faculty_id INTEGER,
        FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
    )
""")

faculties = [
    (301, 'Dr. K Veena devi', 'Associate Professor'),
    (302, 'Dr. Robert Miller', 'Professor')
]
cursor.executemany("INSERT OR REPLACE INTO Faculty VALUES (?, ?, ?)", faculties)

courses = [
    ('DSA0501', 'Query Processing for Data Science', 4, 301),
    ('CS1010', 'Data Structures', 3, 301),
    ('CS2020', 'Operating Systems', 4, 302)
]
cursor.executemany("INSERT OR REPLACE INTO Course VALUES (?, ?, ?, ?)", courses)
conn.commit()

print("--- Faculty Course Allocation ---")
query = """
    SELECT f.faculty_name, f.designation, c.course_id, c.course_name
    FROM Faculty f
    JOIN Course c ON f.faculty_id = c.faculty_id
"""
cursor.execute(query)
for f_name, desig, c_id, c_name in cursor.fetchall():
    print(f"Faculty: {f_name:<18} ({desig}) -> Course: [{c_id}] {c_name}")

conn.close()

if __name__ == "__main__":
    faculty_course_system()

```

Sample Execution Output:

```

--- Faculty Course Allocation ---
Faculty: Dr. K Veena devi    (Associate Professor) -> Course: [DSA0501] Query Processing for
Data Science
Faculty: Dr. K Veena devi    (Associate Professor) -> Course: [CS1010] Data Structures
Faculty: Dr. Robert Miller  (Professor) -> Course: [CS2020] Operating Systems

```

Question 9: Vehicle Registration Database (10 Marks)

Problem Statement: A transport department plans to maintain vehicle registration details in an SQLite database. Design a Vehicle table with suitable attributes and constraints. Write a Python program to connect to the database,

create the table, insert vehicle records, search for a vehicle using its registration number, and display the retrieved information.

1. Requirement Analysis (4 Marks)

- Entities: Vehicle
- Attributes: reg_no (TEXT, PK), owner_name (TEXT), vehicle_type (TEXT), model (TEXT), reg_year (INTEGER).
- Operations: Insert vehicles, Query specific vehicle by reg_no.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: Entity [Vehicle] with primary key reg_no.
- Relational Schema: Vehicle(reg_no [PK], owner_name, vehicle_type, model, reg_year)

3. Table Design & Normalization (4 Marks)

- Fully normalized (3NF compliant).

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```
import sqlite3

def vehicle_registration_system():
    conn = sqlite3.connect("transport.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Vehicle (
            reg_no TEXT PRIMARY KEY,
            owner_name TEXT NOT NULL,
            vehicle_type TEXT,
            model TEXT,
            reg_year INTEGER
        )
    """)

    vehicles = [
        ('TN01AB1234', 'Suresh Kumar', 'Four Wheeler', 'Sedan', 2022),
        ('TN02CD5678', 'Anita Roy', 'Two Wheeler', 'Scooter', 2023),
        ('KA03EF9012', 'Vijay Sharma', 'Four Wheeler', 'SUV', 2021)
    ]
    cursor.executemany("INSERT OR REPLACE INTO Vehicle VALUES (?, ?, ?, ?, ?)", vehicles)
    conn.commit()

    # Search for specific vehicle
    search_reg = 'TN01AB1234'
    print(f"--- Search Result for Vehicle Reg No: {search_reg} ---")
    cursor.execute("SELECT * FROM Vehicle WHERE reg_no = ?", (search_reg,))
    record = cursor.fetchone()

    if record:
        print(f"Reg No: {record[0]}")
        print(f"Owner Name: {record[1]}")
        print(f>Type: {record[2]}")
        print(f>Model: {record[3]}")
        print(f>Year: {record[4]}")
    else:
```

```

        print("Vehicle Not Found!")

    conn.close()

if __name__ == "__main__":
    vehicle_registration_system()

```

Sample Execution Output:

```

--- Search Result for Vehicle Reg No: TN01AB1234 ---
Reg No: TN01AB1234
Owner Name: Suresh Kumar
Type: Four Wheeler
Model: Sedan
Year: 2022

```

Question 10: Hospital Appointment Management System (10 Marks)

Problem Statement: A hospital maintains separate tables for Doctors and Appointments. Design both tables using appropriate primary and foreign keys. Write a Python program to create the tables, insert sample records, and display the doctor name along with the appointment details using a JOIN query.

1. Requirement Analysis (4 Marks)

- Entities: Doctor, Appointment
- Attributes:
 - Doctor: doctor_id (PK), doctor_name, specialization
 - Appointment: appt_id (PK), patient_name, appt_date, doctor_id (FK)
- Relationship: 1 Doctor can have Multiple Appointments.

2. ER Diagram Representation & Logical Schema (4 Marks)

- ER Structure: [Doctor] (1) ----- <Schedules> ----- (N) [Appointment]
- Schema:
 - Doctor(doctor_id [PK], doctor_name, specialization)
 - Appointment(appt_id [PK], patient_name, appt_date, doctor_id [FK])

3. Table Design & Normalization (4 Marks)

- Normalized to 3NF by decoupling doctor info from scheduled appointment rows.

4. Database Connectivity using Python & CRUD Operations (8 Marks)

```

import sqlite3

def hospital_appointment_system():
    conn = sqlite3.connect("hospital_appts.db")
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    cursor.execute("""

```

```

        CREATE TABLE IF NOT EXISTS Doctor (
            doctor_id INTEGER PRIMARY KEY,
            doctor_name TEXT NOT NULL,
            specialization TEXT
        )
    """

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Appointment (
            appt_id INTEGER PRIMARY KEY AUTOINCREMENT,
            patient_name TEXT NOT NULL,
            appt_date TEXT NOT NULL,
            doctor_id INTEGER,
            FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)
        )
    """)

    doctors = [
        (101, 'Dr. Sarah Connor', 'Cardiology'),
        (102, 'Dr. Gregory House', 'Neurology')
    ]
    cursor.executemany("INSERT OR REPLACE INTO Doctor VALUES (?, ?, ?)", doctors)

    appts = [
        ('Michael Scott', '2026-03-30 10:00 AM', 101),
        ('Pam Beesly', '2026-03-30 11:30 AM', 101),
        ('Jim Halpert', '2026-03-31 02:00 PM', 102)
    ]
    cursor.executemany("INSERT INTO Appointment (patient_name, appt_date, doctor_id) VALUES (?, ?, ?)", appts)
    conn.commit()

    print("--- Doctor Appointments Schedule ---")
    query = """
        SELECT d.doctor_name, d.specialization, a.patient_name, a.appt_date
        FROM Doctor d
        JOIN Appointment a ON d.doctor_id = a.doctor_id
    """
    cursor.execute(query)
    for doc_name, spec, pat_name, date in cursor.fetchall():
        print(f"Doctor: {doc_name:<20} ({spec:<12}) | Patient: {pat_name:<15} | Date/Time: {date}")

    conn.close()

if __name__ == "__main__":
    hospital_appointment_system()

```

Sample Execution Output:

```

--- Doctor Appointments Schedule ---
Doctor: Dr. Sarah Connor      (Cardiology  ) | Patient: Michael Scott   | Date/Time: 2026-03-30
10:00 AM
Doctor: Dr. Sarah Connor      (Cardiology  ) | Patient: Pam Beesly       | Date/Time: 2026-03-30
11:30 AM
Doctor: Dr. Gregory House     (Neurology   ) | Patient: Jim Halpert      | Date/Time: 2026-03-31
02:00 PM

```


